Lab 09: Ray Tracing and Path Tracing

CP479 Computer Graphics

October 30, 2026

A rasterizer asks, for every triangle, *which pixels does it cover?* A ray tracer turns the question around and asks, for every pixel, *where did the light that lands here come from?* Follow that light backwards, from the eye to a surface, from the surface to a lamp, off a mirror, through glass, and shadows, reflections and refraction need no tricks at all. Keep following it, bounce after bounce, and you have a path tracer: the whole of global illumination, at the price of noise. Chapter 10 explains both. In this lab you write both, from an empty function to a converged Cornell box, in a few hundred lines of C++.

The lab follows the lessons of Dmitry Sokolov's tinyraytracer. You get the plumbing (a vector type, PNG output, threads, the scenes) and an empty function for every idea in the chapter. You write the functions. Every part prints the numbers that its write-up asks about: time, rays per pixel, triangle tests per ray, and in Part III the error against a reference image. There is no checker and no autograder. The marks are in the write-ups, and several questions only your own measurements can answer. That includes places where your measurement and the chapter will disagree.

An example to look at. The page `https://mastericez.github.io/tiny-raypath-tracer/` renders a Cornell box and the Sponza atrium twice, with a Whitted ray tracer and with a path tracer, and lets you drag sliders through samples per pixel, path depth, and direct against indirect light. It is where Parts I and III are heading, in a bigger renderer. Open it before you start, and again when your own numbers surprise you.

AI coding assistants are permitted and expected. You must be able to explain everything you ship. You must also keep an AI prompt log: for every prompt, record the idea behind it and how you verified the answer (see Section 8). A ray-sphere test and a cosine-weighted sample are among the most copied lines in graphics, so the marks are in the write-ups. In the last ten minutes of the session you will be asked to change one line of your ray tracer, live, and say what the picture will do before you run it.

1 Setup

You need a C++17 compiler and CMake 3.16 or newer. Nothing else: no GPU, no Vulkan, no Python. The first build downloads one header, `stb_image_write.h`, so build once while you are online.

1.1 Get the skeleton and build it

`https://github.com/2110479-Computer-Graphics/Lab09-Raytracing`

Press *Use this template* to make your own copy, then clone that copy. Your work lives in your repository, so commit as you go. Then run *one* of these.

```
make raytrace                                          # macOS, Linux
make.bat raytrace                                      # Windows
cmake -S . -B build && cmake --build build --config Release      # anywhere
```

The three programs are `raytrace`, `mesh` and `pathtrace`, one per part. `make <part>` builds it in Release mode and runs it. Pass flags with `ARGS`:

```
make raytrace ARGS="--depth 2"      # or run it directly: ./build/raytrace --depth 2
make.bat raytrace --depth 2         # Windows takes the flags directly
```

Always time things in a Release build: a Debug build of a ray tracer is ten times slower.

1.2 What the skeleton prints

```
$ make raytrace
  image          800 x 600, vertical fov 60 deg
  depth, eps     4, 0.001
  threads        10
  time           64.9 ms
  rays / pixel   0.000
  saved          out/raytrace.png
  TASK 1a not done
  TASK 1b not done
  TASK 1c not done
  TASK 1d not done
```

Every program prints a panel and saves a PNG in `out/`. The last lines list the TASK functions that are still unfinished: each empty function calls `task_not_done("1b")`, and the program reports it. Delete that line when you write the function. (The optional TASK 2c and 2d are only called, and so only listed, when you ask for the BVH with `--bvh` or `--sweep`.) This is a reminder, not a grade: the picture and the numbers are what tell you whether the function is right. The skeleton's image is one flat sky colour, because `cast_ray` returns the background for every pixel.

2 The skeleton

- **The three files you edit**, and the only ones, all in `student/`: `raytracer.cpp` (TASK 1a–1d), `mesh.cpp` (TASK 2a–2d) and `pathtracer.cpp` (TASK 3a–3d). Each function has a one-line comment; the equations and the steps are in this handout.
- `student/student.h`: the signatures of your functions and three globals the flags set: `EPS` (`--eps`, default 0.001), the distance a new ray starts off the surface it leaves; `MAX_DEPTH` (`--depth`, default 4), the bounce limit of Part I; and `STAGE` (`--stage`), which you may use to switch lessons on and off. Do not edit it.
- `src/core/`: given, and short. Read `vec3.h` (the maths, including `dot`, `cross`, `normalize`, `luminance` and `make_basis`), `ray.h` and `scene.h` (every type you use). `scene.cpp` holds `scene_intersect`, which finds the closest object along a ray by calling your intersection functions, the floor, and the Part I scene. `bvh.cpp` builds the bounding volume hierarchy of Part II, `cornell.cpp` the Cornell box of Part III. `rng.h` is a random number generator; `stats.h` counts rays and tests for the panels.
- `src/raytrace_main.cpp` and its two siblings: one program per part. When a number in a panel surprises you, read the code that printed it.
- `reference/`: `raytrace.png` and `mesh.png`, what a correct solution renders with the default flags; `pathtrace.png`, the Cornell box at 4096 samples per pixel (yours, at 16, is noisier); and `pathtrace_ref.pfm`, the same image as linear floats, which Part III measures your error against. `out/`: your images. Ignored by git.

Program	Flag	Meaning
all	`--threads n`	threads (default: one per core)
	`--out file.png`	where to save, relative to the lab folder
`raytrace`, `mesh`	`--depth n`	bounce limit `MAX_DEPTH` (default 4)
	`--eps x`	`EPS` (default 0.001)
`raytrace`	`--stage n`	sets `STAGE` (default 99: everything)
	`--probe`	the chapter's two ray-sphere examples, through your code
`mesh`	`--tris 1k`	knot size: `1k 2k 4k 16k 64k 256k` triangles
	`--obj f.obj`	your own mesh instead of the knot
	`--flat`	drop the vertex normals
	`--glass`	make the mesh glass
	`--bvh`	use your `intersect_bvh` (TASK 2d)
	`--split sah`	BVH split: `sah` (default) or `median`
	`--sweep`	time primary rays against 1k to 256k triangles
`pathtrace`	`--spp n`	samples per pixel (default 16)
	`--depth n`	bounce limit (default 64)
	`--no-nee`	no light sampling (TASK 3b off)
	`--no-rr`	no Russian roulette (TASK 3c off)
	`--eps x`, `--seed n`	as above; another seed gives other noise
	`--sweep`	error and time at 1, 4, 16, 64, 256 spp

Table 1: Flags. `make raytrace ARGS="--depth 2 --eps 0"` passes two; on Windows, `make.bat raytrace --depth 2 --eps 0`.

3 Part I: A tiny ray tracer

Run `make raytrace`. The scene is four spheres (ivory, red rubber, glass and a mirror) over a checkerboard floor, lit by three point lights, seen by a pinhole camera at the origin that looks down $-z$. You write the camera, the intersection, the two bending rules and the shading, in that order.

The ray (EQ-10.01). A ray is a half line

$$\mathbf{r}(t) = \mathbf{o} + t\,\mathbf{d}, \qquad t > 0, \tag{1}$$

from the origin $\mathbf{o}$ along the unit direction $\mathbf{d}$. The camera ray through pixel (x, y) of a $W \times H$ image with vertical field of view φ starts at the origin and points at

$$\Big(\big(\tfrac{2(x+0.5)}{W} - 1\big)\tan\tfrac{\varphi}{2}\,\tfrac{W}{H},\ -\big(\tfrac{2(y+0.5)}{H} - 1\big)\tan\tfrac{\varphi}{2},\ -1\Big), \tag{2}$$

normalized. Row 0 is the top of the image and $+y$ is up, hence the minus sign.

Ray-sphere (EQ-10.02). Put Equation 1 into $|\mathbf{p} - \mathbf{c}|^2 = R^2$. With $\mathbf{oc} = \mathbf{o} - \mathbf{c}$:

$$at^2 + bt + c = 0, \qquad a = \mathbf{d}\cdot\mathbf{d},\ b = 2\,\mathbf{d}\cdot\mathbf{oc},\ c = \mathbf{oc}\cdot\mathbf{oc} - R^2, \qquad t = \frac{-b \mp \sqrt{b^2 - 4ac}}{2a}. \tag{3}$$

A negative discriminant misses. Take the near root if it is larger than `EPS`, otherwise the far root (the ray starts inside the sphere, which every ray refracted into the glass does), otherwise there is no hit.

Reflection and refraction (EQ-10.04, Snell). With a unit normal $\mathbf{n}$ facing the incoming ray, $\cos\theta_i = -\mathbf{d}\cdot\mathbf{n}$ and $\eta = n_1/n_2$:

$$\mathbf{r} = \mathbf{d} - 2(\mathbf{d}\cdot\mathbf{n})\,\mathbf{n}, \qquad \mathbf{t} = \eta\,\mathbf{d} + \big(\eta\cos\theta_i - \sqrt{k}\big)\mathbf{n}, \quad k = 1 - \eta^2(1 - \cos^2\theta_i). \tag{4}$$

$k < 0$ is total internal reflection: there is no refracted ray. Entering glass, $\eta = 1/1.5$. Leaving it, the normal you were given points the wrong way: flip it and use $\eta = 1.5$.

Shading. At a hit point $\mathbf{p}$ with normal $\mathbf{n}$, for each light of intensity I in direction $\mathbf{l}$ that a *shadow ray* from $\mathbf{p}$ reaches unblocked, Lambert and Phong add

$$D \mathrel{+}= I \max(0, \mathbf{l} \cdot \mathbf{n}), \qquad S \mathrel{+}= I \max\big(0, -\text{reflect}(-\mathbf{l}, \mathbf{n}) \cdot \mathbf{d}\big)^{e}. \tag{5}$$

The colour is a weighted sum with the material's four `albedo` weights: $\mathbf{k}_d D a_0 + S a_1 + L_{\text{reflected}} a_2 + L_{\text{refracted}} a_3$, where the last two are `cast_ray` called again, one bounce deeper (§10.1.5, Figure F-10.01). This is Whitted's ray tracer: one recursive function.

3.1 TASK 1: `student/raytracer.cpp`

Listing 1: The four TASKs of Part I

```
Ray  camera_ray(int x, int y, int width, int height, float fov);   // 1a
bool ray_sphere(const Ray& r, const Sphere& s, float& t);          // 1b
vec3 reflect(const vec3& d, const vec3& n);                        // 1c
bool refract(const vec3& d, const vec3& n, float ior, vec3& out);  // 1c
vec3 cast_ray(const Ray& r, const Scene& scene, int depth);        // 1d
```

TASK 1a is Equation 2, 1b Equation 3, 1c Equation 4 and 1d Equation 5. `cast_ray` finds the closest hit with the given `scene_intersect(r, scene, hit)`, which calls your `ray_sphere` for every sphere and fills in `hit.t`, `hit.p`, `hit.n` (the unit normal pointing *out* of the object) and `hit.material`. Every new ray (shadow, reflected, refracted) starts at `hit.p` moved `EPS` along the normal, to the side the new ray goes. A shadow ray is blocked if it hits something closer than the light. Return `scene.background` when `depth > MAX_DEPTH` or nothing is hit.

Build `cast_ray` one lesson at a time, as tinyraytracer does, and look at the picture after each one (Figure 1): a flat colour per object, then diffuse light, specular highlights, shadows, reflection, refraction. Keep each lesson behind `if(STAGE >= n)` and `--stage n` will show any of them again later; the write-ups need that. `make raytrace ARGS=--probe` runs two worked examples from the chapter through your `ray_sphere` alone.

3.2 Expected result

```
$ make raytrace
  image          800 x 600, vertical fov 60 deg
  depth, eps     4, 0.001
  threads        10
  time           20.4 ms
  rays / pixel   8.800
  saved          out/raytrace.png
```

Rays per pixel counts every call to `scene_intersect`: camera rays, shadow rays and every reflected and refracted ray, divided by the pixel count. Yours may differ from 8.8 if you skip rays the solution casts. Say which, in write-up 1.3.

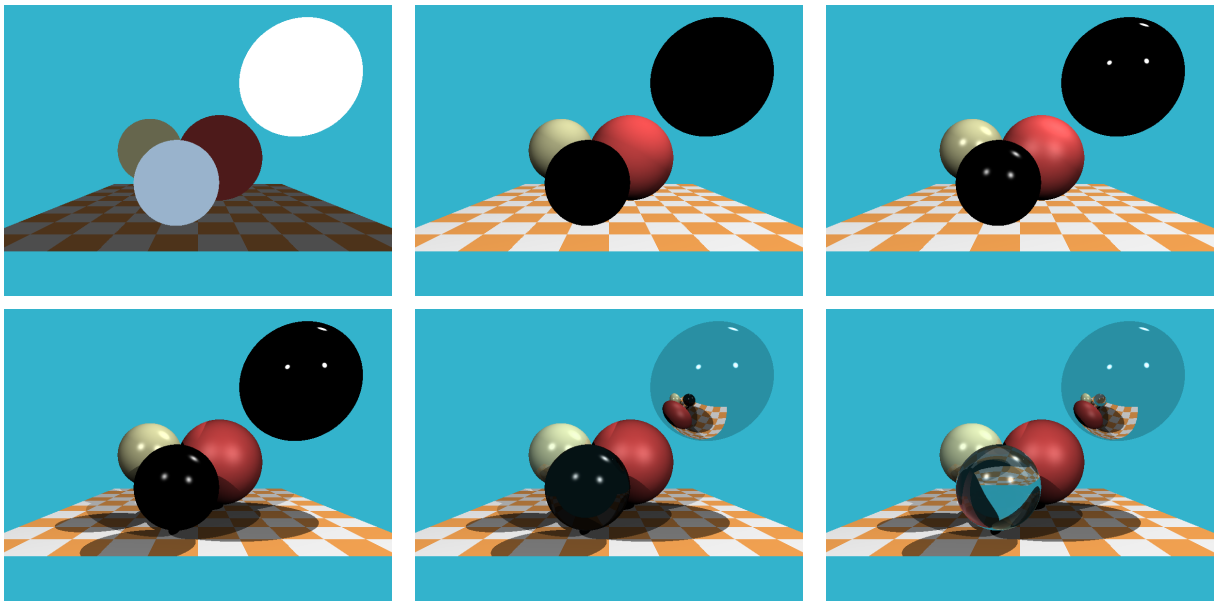


Figure 1: `--stage 1` to `6`. **1**: the diffuse colour of whatever is hit. **2**: Lambert, $\mathbf{k}_d D a_0$; glass and mirror have $a_0 = 0$ and turn black. **3**: Phong highlights. **4**: shadow rays. The glass sphere casts a full shadow: a shadow ray cannot see through it. **5**: reflection; the mirror shows the scene, the glass a faint reflection. **6**: refraction; the final image, `reference/raytrace.png`. Light brighter than 1 is scaled down by its largest channel when saved, so the floor's checks look orange and white, not dark.

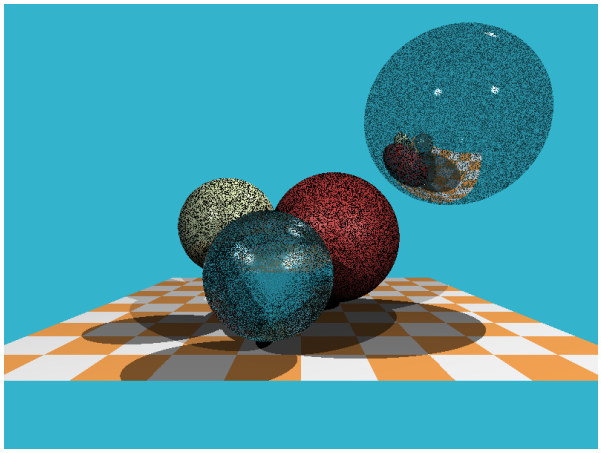


Figure 2: `--eps 0`. Write-up 1.2 asks what this is, and why the floor is spared.

Write-up 1.

1. Run `--probe`. The chapter's worked example for EQ-10.02 (§10.1.2: ray from $(0, 0, 0)$ along $(0, 0, -1)$, sphere at $(0, 0, -5)$, $R = 1$) computes $b = 10$ and concludes that neither root is valid. What does your `ray_sphere` say? Redo the example by hand with Equation 3, find the slip, and give both roots. Then check the chapter's second example (the ray from $(0, 0, 3)$) the same way.
2. Run `--eps 0` and explain Figure 2 at the level of floating point: where does a shadow ray start, and what does it hit first? The floor is the one surface without speckles: read how `scene.cpp` intersects it, and say why. `EPS` is a *length*: if this scene were modelled in millimetres (every coordinate times 1000), what would 0.001 do?

3. Rays per pixel for `--depth 0, 1, 2, 3, 4` in a table, and the time of each. The chapter's ray tree (Figure F-10.01) grows $1 \to 3 \to 9$. Your numbers grow differently: explain them from what your `cast_ray` casts at each hit (how many shadow rays? does it cast reflected and refracted rays whose weight a_2 or a_3 is zero?) and from rays that leave the scene. Which rays could you skip without changing a single pixel, and what would it save?
4. Inside the glass sphere there is a dark wedge (Figure 1, stage 6). Make it go away and come back by changing one line, and say what it is, using Equation 4. Where does that light really go, and what does this ray tracer do with it instead?

4 Part II: Triangles

Run `make mesh`. The same scene with a gold torus knot in the top left corner: 1024 triangles by default (`--tris`), or any OBJ file (`--obj`). Your `cast_ray` renders it unchanged. All you add is the intersection with triangles.

Möller-Trumbore (DEF-10.01). A point inside triangle $\mathbf{v}_0\mathbf{v}_1\mathbf{v}_2$ is $(1-u-v)\mathbf{v}_0+u\mathbf{v}_1+v\mathbf{v}_2$ with $u, v \geq 0$ and $u+v \leq 1$. Setting it equal to Equation 1 gives three linear equations in (t, u, v), which Cramer's rule solves with two cross products:

$$\mathbf{e}_1 = \mathbf{v}_1 - \mathbf{v}_0, \quad \mathbf{e}_2 = \mathbf{v}_2 - \mathbf{v}_0, \quad \mathbf{h} = \mathbf{d} \times \mathbf{e}_2, \quad a = \mathbf{e}_1 \cdot \mathbf{h}, \quad \mathbf{s} = \mathbf{o} - \mathbf{v}_0, \quad \mathbf{q} = \mathbf{s} \times \mathbf{e}_1,$$
$$u = \frac{\mathbf{s} \cdot \mathbf{h}}{a}, \qquad v = \frac{\mathbf{d} \cdot \mathbf{q}}{a}, \qquad t = \frac{\mathbf{e}_2 \cdot \mathbf{q}}{a}. \tag{6}$$

$|a| \approx 0$ means the ray is parallel to the triangle. The same u and v interpolate anything stored at the corners. Here: the vertex normals, which make 1024 flat triangles look like a smooth tube.

The bounding volume hierarchy (§10.2), optional. Testing every triangle costs $O(N)$ per ray. The given `build_bvh` (in `src/core/bvh.cpp`) puts the triangles in a binary tree of axis-aligned boxes, and a ray that misses a box skips everything inside it, so a ray visits $O(\log N)$ nodes. A box is tested with the slab method (EQ-10.03):

$$t_{k,j} = (\mathrm{box}_{k,j} - o_k)/d_k, \qquad t_{\mathrm{entry}} = \max_k \min(t_{k,0}, t_{k,1}), \qquad t_{\mathrm{exit}} = \min_k \max(t_{k,0}, t_{k,1}),$$
$$\text{hit} \iff t_{\mathrm{entry}} \leq t_{\mathrm{exit}} \text{ and } t_{\mathrm{exit}} > 0. \tag{7}$$

The builder chooses each split either at the median or by the surface area heuristic, EQ-10.05: $C = C_T + \frac{SA(L)}{SA(P)} N_L C_I + \frac{SA(R)}{SA(P)} N_R C_I$, with the chapter's $C_T = 1$, $C_I = 1.5$. Writing the traversal is optional. Brute force is a complete answer to Part II. The BVH is how you find out what $O(\log N)$ means on your machine.

One sphere first. `scene_intersect` first tests each mesh's bounding sphere with *your* `ray_sphere`, and skips every triangle of a mesh whose sphere the ray misses: a one-node BVH. That is why brute force is usable at all, and why *tests per ray* is less than N.

4.1 TASK 2: `student/mesh.cpp`

Listing 2: Part II. 2a and 2b are required, 2c and 2d optional

```
// 2a
bool ray_triangle(const Ray& r, const vec3& v0, const vec3& v1, const vec3& v2,
                  float& t, float& u, float& v);
// 2b
bool intersect_mesh(const Ray& r, const Mesh& m, float t_max, Hit& hit);
// 2c
```

```
bool ray_aabb(const Ray& r, const vec3& inv_d, const AABB& box, float t_max,
              float& t_entry);
// 2d
bool intersect_bvh(const Ray& r, const Mesh& m, float t_max, Hit& hit);
```

`ray_triangle` accepts a hit only at $t >$ `EPS`. `ray_aabb` writes t_{entry} and also misses a box that starts beyond `t_max`. `intersect_mesh` tests every triangle of `m.tris` (each holds three indices into `m.positions`) and calls `count_triangle_test()` once per test, keeps the closest hit below `t_max`, and fills in `hit.t`, `hit.tri` and `hit.n`: the vertex normals interpolated with u, v when `m.normals` is not empty, else the face normal $\mathbf{e}_1 \times \mathbf{e}_2$, normalized. `intersect_bvh` does the same through `m.bvh`: a stack of node indices, `count_node_visit()` per node, skip a node whose box is missed or farther than the best hit so far, test a leaf's triangles, and push an inner node's two children, the farther one first (test both children's boxes to know which). A node with `count > 0` is a leaf holding the triangles `m.bvh.order[first]` to `order[first + count - 1]`; any other node's children are `nodes[index + 1]` and `nodes[right]`, and node 0 is the root. With `--bvh` the program uses it, if 2c and 2d are written.

4.2 Expected result

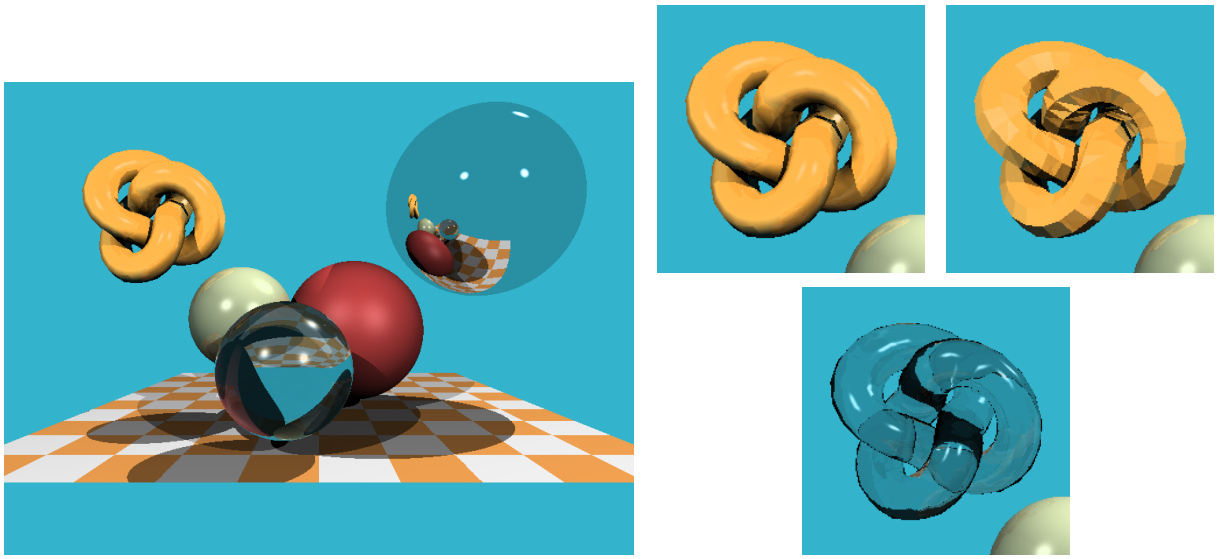


Figure 3: **Left:** `make mesh`, `reference/mesh.png`. The knot is in the mirror too, for free. **Right:** the default knot, the same triangles with `--flat`, and with `--glass`, which gives it the glass sphere's material. Through glass, light crosses the tube twice per side.

```
$ make mesh ARGS="--bvh"
  mesh            torus knot, 1024 triangles, smooth normals
  bvh             sah split: 1111 nodes, 556 leaves, depth 27, built in 0.4 ms
  threads         10
  time            246.2 ms
  rays / pixel    10.917
  tests / ray     3.5 triangle tests
  nodes / ray     14.4 BVH nodes visited
$ make mesh ARGS="--sweep"
  sweep           primary rays only, 320 x 240, the knot alone
  triangles  method          build ms   render ms   tests/ray  nodes/ray
       1024  brute force          0.0        26.5       322.0        0.0
       1024  bvh median           0.2         2.4         3.2        8.8
       1024  bvh sah              0.4         2.9         2.6       14.0
  ...
```

Without `--bvh` the same render tests every triangle of the knot for every ray that reaches its bounding sphere. It took 1.3 s on the machine above (10 threads) and will take several seconds on yours. Larger knots scale accordingly: that is the point of the sweep, which skips brute force above 16k triangles. Run one larger size by hand if you are patient.

Write-up 2. Write your predictions for question 1 in the report *before* you run `--sweep`.

1. The sweep, as a table and as a log-log plot of render time and of triangle tests per ray against N. Fit the slopes. Brute force's tests per ray is not N: compute what fraction of the rays reach the knot's bounding sphere. The chapter says a BVH over 10M triangles needs about 30 operations per ray, and its lab plan asks for "10K+ triangles in under 5 ms per image". Extrapolate your brute force (and your BVH, if you wrote it) to 10^7 triangles, and say how far the 16k knot is from 5 ms on your machine, at 320×240 and at 800×600.
2. `--flat` against the default: the same triangles, a different picture. Explain from Equation 6 why u and v are "free", and what else you could interpolate with them.
3. Back-face culling: change your test from $|a| < 10^{-8}$ to $a < 10^{-8}$ and render the default knot. What changes, and why? Which way do the knot's triangles face? Now render it with `--glass`, with and without culling, and explain the difference. Can a GPU cull the triangles of a glass object?
4. *Optional, if you wrote the BVH.* Median against SAH in the sweep: build time, nodes per ray, triangle tests per ray, render time. In this builder the SAH tree visits *more* nodes than the median one but tests fewer triangles. Explain that from EQ-10.05 and the costs C_T, C_I in `bvh.cpp`, and say which one wins on your machine and by how much.

5 Why path tracing, not only ray tracing?

Reading only: nothing to hand in. Part I is a complete ray tracer, and its pictures look convincing: shadows, a mirror, glass. Part III throws most of that machinery away and replaces it with random numbers and noise. This section explains why that trade is worth making.

What Whitted's ray tracer computes. At every hit, Part I's `cast_ray` looks in a handful of directions it can name in advance: towards each point light (a shadow ray), the mirror direction and the refracted direction. Light that arrives from anywhere else is ignored. That is exact for two special cases only: a point light, which sends light from a single direction, and a perfect mirror or glass surface, which passes light along a single direction. Everything else in the picture is an approximation, and some of it is simply missing:

- **Light from other surfaces.** A diffuse wall lit by the lamp is itself a light source for everything that sees it. Whitted's tracer never asks: a shadow is pitch black unless an *ambient* constant is added, the same everywhere, and the red wall never tints the white box beside it (*colour bleeding*). The ceiling of a Cornell box, which the lamp cannot see, stays dark.
- **Soft shadows.** A real lamp has an area, so a point near a shadow's edge sees part of it. A shadow ray towards one point is either blocked or not: the edge is razor sharp.
- **Light through glass onto a diffuse surface** (*caustics*). The floor's shadow ray towards the lamp hits the glass sphere and counts as blocked, so the glass casts a black shadow instead of a bright focused spot.
- **Anything that is not a perfect mirror**: brushed metal, glossy paint, frosted glass. They scatter light over a range of directions, not into one.

What path tracing changes. All of these are the same thing: the integral of the rendering equation (Equation 8 in the next section) over *every* incoming direction, where light can arrive from any surface, not only from a light. Whitted replaced the integral by the few directions he could name. Cook et al. (1984) sampled some of them at random (*distributed ray tracing*: soft

shadows, glossy reflection, motion blur). Kajiya (1986) showed that sampling all of it at random, one bounce after another, solves the whole equation: that is path tracing. One algorithm, with no special cases, produces every effect above, and its average converges to the physically correct image.

Effect	**Whitted ray tracing (Part I)**	**Path tracing (Part III)**
direct light, mirror, refraction	yes	yes
shadows	hard, from point lights	soft, from area lights
light bounced off other surfaces	no: a constant ambient term	yes, every bounce
colour bleeding	no	yes
caustics (light through glass)	no: a black shadow	yes (noisy)
glossy surfaces	no	yes, with a glossy material
noise	none	yes, $\sigma \propto 1/\sqrt{N}$
rays per pixel	a few	thousands, offline
converges to the right answer	no	yes (unbiased)

Table 2: What each method computes.

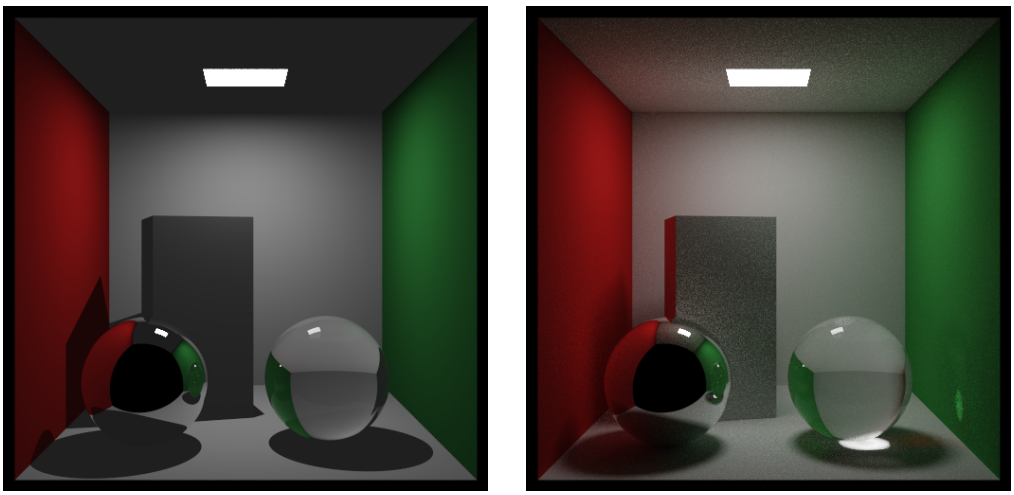


Figure 4: The same Cornell box, with a mirror and a glass sphere, from the example page. **Left: Whitted ray tracing.** Hard shadows, a black shadow under the glass, a dark ceiling, and a flat ambient term filling the shadows. **Right: path tracing, 1024 samples per pixel.** Soft shadows, red and green light on the box and the floor, a lit ceiling, and a bright caustic under the glass sphere. Both took the same scene; only the integrator changed.

The price, and why it is now paid in real time. A ray tracer's image is finished after one pass. A path tracer's is an average of random samples, so it is noisy, and its noise falls only as $1/\sqrt{N}$: ten times less noise costs a hundred times the work. For decades that confined path tracing to film, at thousands of samples per pixel and minutes per frame. The rest of Chapter 10 is the story of paying less: sampling where the light is (importance sampling and next event estimation, which you write in Part III), stopping paths early without bias (Russian roulette), reusing samples across pixels and frames (ReSTIR, §10.5), hardware that traces rays fast (RT cores, §10.6.1), and denoisers that clean up an image of one to four samples (§10.6.2). Together they are why a game such as *Cyberpunk 2077* can path trace in real time on a high-end GPU. Part III's Cornell box at 16 samples per pixel, next to the 4096-sample reference, is exactly the gap those techniques close. The example page's *What changes* section shows the difference interactively.

6 Monte Carlo sampling

Part III computes, at every pixel, an integral that has no closed form, and it does so by averaging random samples. This section is the theory behind every random number the path tracer draws. Read it before Part III: write-up 3 measures each idea in it.

What is computed. The light leaving a point $\mathbf{x}$ towards the eye is the light the point emits plus the light it reflects, gathered from every direction ω of the hemisphere above it (the rendering equation):

$$L_o(\mathbf{x}) = L_e(\mathbf{x}) + \int_\Omega f\, L_i(\mathbf{x},\omega)\cos\theta\, d\omega, \qquad f = \frac{\rho}{\pi} \text{ for a diffuse (Lambert) surface.} \tag{8}$$

θ is the angle between ω and the normal and ρ is the albedo. The incoming light L_i is itself the L_o of whatever point lies in direction ω, so the integral nests without end. Part I's ray tracer replaces it with a few chosen directions: towards each light, and the mirror and refracted directions. A path tracer estimates it.

6.1 The Monte Carlo estimator

To estimate $I = \int f(x)\, dx$, draw N random samples x_k with a probability density p and average:

$$I \;\approx\; \hat{I}_N = \frac{1}{N}\sum_{k=1}^{N}\frac{f(x_k)}{p(x_k)}, \qquad \sigma\big[\hat{I}_N\big] = \frac{\sigma\big[f/p\big]}{\sqrt{N}}. \tag{9}$$

Dividing by p is what makes it right on average: $E\big[f(X)/p(X)\big] = \int \frac{f(x)}{p(x)}\, p(x)\, dx = I$, for any p that is nonzero wherever f is (EQ-10.06). Such an estimator is *unbiased*: its error is pure noise, and noise averages away. The noise is the standard deviation σ, and it falls as $1/\sqrt{N}$: four times the samples halve it. There are two ways to less noise, and Part III uses both: more samples, and a density p shaped like f, which makes f/p nearly constant. If p were exactly proportional to f, every sample would return I and the noise would be zero.

Part III's panel measures both properties. *RMSE* against a 4096-sample reference is the noise σ of your image (plus a little of the reference's own), and *mean* checks "right on average".

6.2 Sampling the pixel

A pixel's colour is the average of the light over its square: an integral too. `pathtrace` sends each sample through a uniformly random point of the pixel ($p = 1$ over the unit square), so averaging the samples estimates it with Equation 9, and anti-aliasing comes for free. It is also why the lamp's edge is the noisiest part of the image: a pixel that the edge crosses returns 15 or 0.1 depending on where its sample lands. Part I sends one ray through each pixel's centre instead: no noise, and jagged edges.

6.3 Sampling a bounce: cosine-weighted directions

At a diffuse surface the integrand of Equation 8 is $\frac{\rho}{\pi} L_i \cos\theta$. Directions near the horizon, where $\cos\theta \approx 0$, contribute little, and sampling the hemisphere uniformly ($p = \frac{1}{2\pi}$) spends as many rays on them as on the useful ones. Sample instead with

$$p(\omega) = \frac{\cos\theta}{\pi}, \qquad \frac{\frac{\rho}{\pi} L_i \cos\theta}{p(\omega)} = \rho\, L_i. \tag{10}$$

The cosine and the π cancel: each bounce multiplies the path's *throughput* by the albedo ρ, and that is all. ($\int_\Omega \frac{\cos\theta}{\pi}\, d\omega = 1$, so it is a density.) This is *importance sampling* (§10.3.2), its cheapest kind.

To draw such a direction from two uniform numbers $u_1, u_2 \in [0, 1)$, pick a point uniformly on the unit disk and lift it straight up onto the hemisphere (Malley's method, Figure 5a):

$$r = \sqrt{u_1}, \quad \phi = 2\pi u_2, \qquad \omega_{\text{local}} = \big(r\cos\phi,\ r\sin\phi,\ \sqrt{1-u_1}\big). \tag{11}$$

The square root makes the disk uniform: the area inside radius r is πr^2, a fraction r^2 of the disk, so r^2 must be the uniform number. Lifting turns uniform on the disk into proportional to $\cos\theta$, because a patch of the hemisphere at angle θ looks $\cos\theta$ times smaller seen from straight above. The local z axis is the normal: `make_basis(n, t, b)` gives the other two axes, and the direction in the world is $x\,\mathbf{t} + y\,\mathbf{b} + z\,\mathbf{n}$.

6.4 Sampling the lamp: from directions to area

The lamp covers a tiny part of the hemisphere. A cosine-sampled ray hits it rarely, and when it does, that one sample carries all of the light: noise (`--no-nee` shows how much). Since we know where the lamp is, integrate over its area instead of over directions. A patch dA of the lamp at distance d, tilted by θ_l away from the line to $\mathbf{x}$, covers the solid angle $d\omega = \cos\theta_l\, dA/d^2$ (Figure 5b), so the light arriving straight from the lamp is

$$L_{\text{direct}}(\mathbf{x}) = \int_{\text{lamp}} \frac{\rho}{\pi}\, L_e\, \frac{\cos\theta_x \cos\theta_l}{d^2}\, V\, dA \;\approx\; \frac{\rho}{\pi}\, L_e\, \frac{\cos\theta_x \cos\theta_l}{d^2}\, V\, A, \tag{12}$$

where the visibility V is 1 if nothing blocks the segment from $\mathbf{x}$ to the lamp point $\mathbf{q}$, and 0 otherwise. The approximation is one sample of Equation 9: $\mathbf{q}$ uniform on the lamp has $p = 1/A$, and dividing by p multiplies by A. This is *next event estimation* (NEE): at every bounce, ask the lamp directly. It splits the light into two parts that must not overlap. The direct light comes from NEE at every bounce, so a bounce ray that happens to hit the lamp must add nothing; only the camera ray (bounce 0), which no NEE came before, may see the lamp itself. Weighting the two strategies instead of choosing one is multiple importance sampling (§10.3.3), which this lab leaves out.

6.5 Russian roulette: stopping without bias

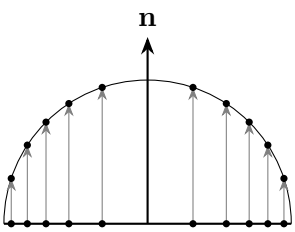
A path could bounce forever, and each bounce adds less, since its throughput shrinks by ρ every time. Cutting it at a fixed depth throws the rest away: the estimate is biased, too dark (write-up 3.4). Russian roulette stops it at random instead (EQ-10.08): with probability q the path ends, and if it survives, its throughput is divided by $1 - q$. Whatever F the rest of the path would have returned, the expected value is unchanged:

$$E = (1-q)\,\frac{F}{1-q} + q \cdot 0 = F. \tag{13}$$

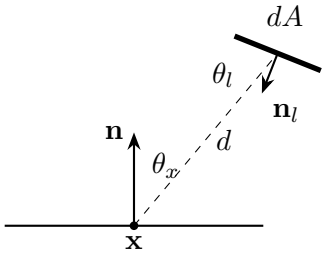
The mean stays right, and the price is noise: a few survivors carry boosted weights. To pay it where it is cheap, stop the paths that matter least. This lab uses $q = 1 - \text{luminance(throughput)}$, clamped to $[0.05, 0.95]$, from the fourth surface on (review question 19).

6.6 One path is one sample

Expanding Equation 8 with N samples at every bounce would cost $1, N, N^2, \ldots$ rays: the ray tree of Figure F-10.01. A path tracer takes *one* sample per bounce, so each camera sample follows a single chain. At every surface it adds NEE times the throughput so far, multiplies the throughput by ρ for the next cosine sampled direction, and lets Russian roulette decide whether to go on. Each chain is a noisy but unbiased estimate of the whole nested integral, and the pixel averages `--spp` of them.



(a) equal areas of the disk (dots below), lifted: denser where $\cos\theta$ is large



(b) $d\omega = \cos\theta_l \, dA/d^2$

Figure 5: **(a)** Malley's method, seen from the side: points spread evenly over the disk's area, lifted onto the hemisphere, crowd near the normal with density $\cos\theta/\pi$. **(b)** A patch of the lamp seen from $\mathbf{x}$: the farther and the more tilted it is, the smaller the solid angle it covers.

7 Part III: A path tracer

Run `make pathtrace`. The Cornell box: a red wall on the left, a green one on the right, two white boxes, and a small lamp in the ceiling, all made of 36 triangles that your Part II code intersects. Every surface is diffuse. A ray tracer would light it with shadow rays and stop. A path tracer keeps going: the light that the red wall reflects onto the boxes is light too, and so is the light the boxes reflect onto the ceiling, which the lamp itself never reaches.

Section 6 explains every random number it draws. The example page (`https://mastericez.github.io/tiny-raypath-tracer/`) runs the same experiments on a Cornell box with a mirror and a glass sphere: its sections *Monte Carlo convergence*, *Path depth* and *Direct vs. indirect light* are write-ups 3.1 and 3.4 seen side by side.

7.1 TASK 3: `student/pathtracer.cpp`

Listing 3: Part III

```
vec3 cosine_sample_hemisphere(const vec3& n, float u1, float u2);     // 3a
vec3 sample_light(const Hit& hit, const PtScene& scene, Rng& rng);  // 3b
bool russian_roulette(vec3& throughput, int bounce, Rng& rng);      // 3c
vec3 path_trace(const Ray& r, const PtScene& scene, Rng& rng,       // 3d
                const PtOptions& opt);
```

`pt_intersect(ray, scene, t_max, hit)` finds the closest triangle (with your `intersect_mesh`) and `pt_material(scene, hit)` its albedo and emission. `rng.next()` is a uniform number in $[0, 1)$.

- **3a**: Equation 11, turned into world axes with `make_basis`.
- **3b**: one sample of Equation 12. Pick $\mathbf{q} = \text{corner} + u_1\mathbf{e}_1 + u_2\mathbf{e}_2$ with u_1, u_2 from `rng.next()`. The shadow ray starts at $\mathbf{o} = \mathbf{x} + \varepsilon\mathbf{n}$: aim it at $\mathbf{q}$ *from* $\mathbf{o}$ and measure d from $\mathbf{o}$. Aimed from $\mathbf{x}$, it meets the lamp itself just before d, and the lamp shadows its own light (we fell into this one). Return 0 if $\cos\theta_x = \mathbf{l}\cdot\mathbf{n} \le 0$, if $\cos\theta_l = -\mathbf{l}\cdot$`light.normal()` ≤ 0, or if `pt_intersect` finds anything closer than $d - \varepsilon$. ρ is `pt_material(scene, hit).albedo`.
- **3c**: Section 6.5. `bounce` counts the surfaces hit so far, from 0: before bounce 3 always continue.
- **3d**: throughput $= (1, 1, 1)$, result $= 0$. For each bounce up to `opt.max_depth`: `pt_intersect` the ray (`t_max` is `INFINITY`) and stop on a miss; flip `hit.n` to face the ray. If the triangle is the lamp (emission > 0), add throughput $\times$ emission when bounce is 0 or NEE is off, but only if the ray hit the lamp's front, $\mathbf{d}\cdot$`light.normal()` < 0;

then stop. The lamp shines downwards: behind it, in the gap under the ceiling, rays would collect light that NEE never counts (the other trap). Otherwise, with NEE add throughput $\times$ `sample_light`; continue from $\mathbf{x} + \varepsilon\mathbf{n}$ along `cosine_sample_hemisphere(hit.n, ...)`, multiply the throughput by ρ, and with Russian roulette on, stop when `russian_roulette` says so.

The program renders 256×256 pixels at `--spp` samples each and compares the result with `reference/pathtrace_ref.pfm`, the same box at 4096 samples per pixel:

```
$ make pathtrace
  scene           Cornell box, 36 triangles, 256 x 256
  options         depth 64, next event estimation on, Russian roulette on, eps 0.001
  threads         10
  spp             16
  time            412.4 ms
  rays / sample   5.04
  mean            0.1582  (reference 0.1584)
  RMSE            0.0206  against the 4096 spp reference, on the 99.2% of the pixels away from the lamp
```

mean is the average luminance of the image: a biased renderer gets it wrong however many samples it takes. *RMSE* is the root mean square difference from the reference, which is the noise, over every pixel except the lamp and its edge. There, each pixel is either lamp (15) or ceiling (0.1) depending on where its random sample lands, and those 0.7% of the pixels would otherwise make up nine tenths of the error. `--sweep` renders 1, 4, 16, 64 and 256 samples per pixel and fits the slope of $\log \text{RMSE}$ against $\log \text{spp}$. Every sample has its own fixed random seed, so a render is the same every time; `--seed 1` gives different noise.

7.2 Expected result

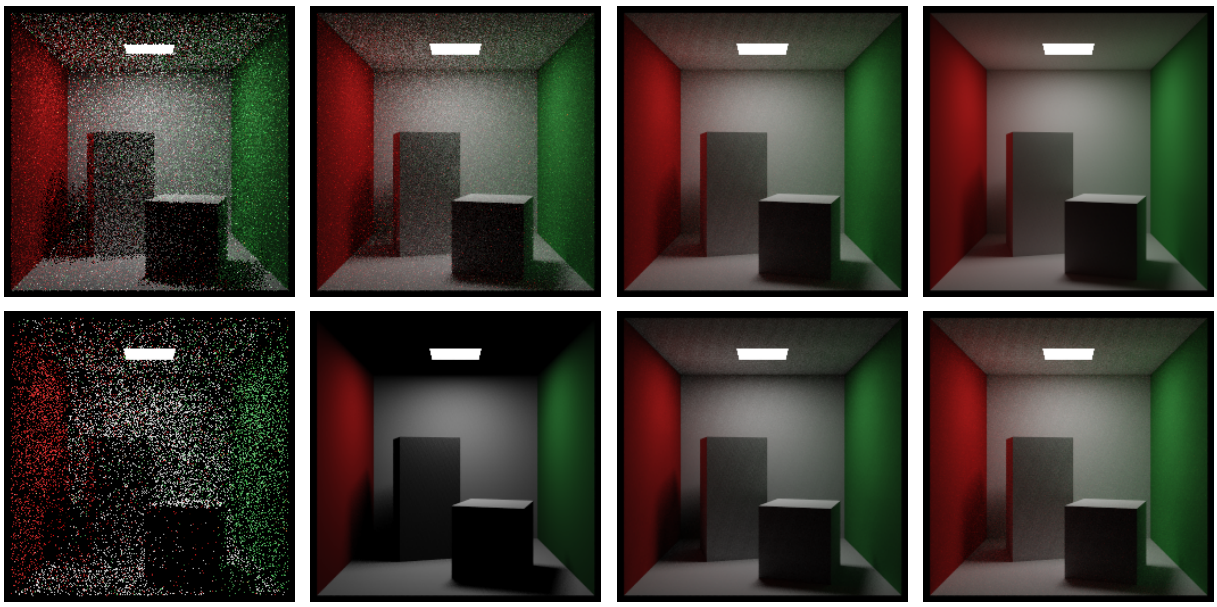


Figure 6: **Top:** 1, 16 and 256 samples per pixel, and the 4096-sample reference. **Bottom:** 16 samples with `--no-nee`; then 256 samples with `--depth 1`, `2` and `10`. Depth 1 is direct light only: the ceiling, which faces away from the lamp, is black, and so are the shadows. The red on the tall box's left side appears at depth 2.

Write-up 3. Write your predictions for questions 1 and 2 in the report *before* you run them.

1. `--sweep`: the table and a log-log plot of RMSE against spp. Is the slope $-\frac{1}{2}$? Look at the column RMSE$\times\sqrt{\text{spp}}$: why does it creep up at 256 samples? (What is the reference made of? Run `--sweep --seed 1` too: does it creep the same way?) How many samples per

pixel would make the image 10× less noisy than at 16, and how long would that take on your machine? Compare with the chapter's "4× samples = 2× less noise".

2. `--no-nee` at 16 samples per pixel: RMSE and time against the default, and the pictures. How many samples does the no-NEE renderer need to match NEE at 16? Explain the gap from the size of the lamp: roughly what fraction of cosine-weighted directions from the middle of the floor hit it?

3. Russian roulette: `--no-rr` against the default at 64 samples: mean, RMSE, time, rays per sample. Is it unbiased? It adds noise: is it worth it? Compare the efficiency $1/(\text{RMSE}^2 \times \text{time})$ of both. Then read the chapter's Part 1 code (Lab 10, *Part 1: CPU Path Tracer*). It does not compile as printed: what is missing from `Hit`? Can `return hit.emissive ? vec3(12.f) : vec3(0)` after the roulette ever return 12? It computes q from the albedo of the current hit, not from the throughput that §10.3.4 and review question 19 describe, and plays from the first bounce: which of these make the image *wrong*, and which only make it noisier or slower? Measure at least one by changing your code.

4. `--depth 1, 2, 3, 5, 10` at 64 samples: the mean of each against the reference. How much light does each extra bounce add, and how much does a hard cut at depth 10 (the chapter's `if (depth > 10)`) lose here? Why does Russian roulette with `--depth 64` not lose it, though most paths stop long before 64?

5. The two traps of TASK 3b and 3d. Put one of them back in your code: report the mean with and without it, show the picture, and explain why it changes the mean, using Equation 12.

8 Submission

Submit your `student/` directory and a report, in any format you like (Google Docs, Word, PDF), containing:

- **Three source files**: `raytracer.cpp`, `mesh.cpp` and `pathtracer.cpp`. Nothing else in the repository should have changed.
- **Your machine**: CPU, number of threads, compiler. Every number you report is about your machine.
- **Predictions**: for the Part II sweep and Part III questions 1 and 2, written in the report *before* you ran the measurement, and left as you wrote them. A wrong prediction costs nothing. A missing one, or one written afterwards, does.
- **Images and panels**: `out/raytrace.png`, `out/mesh.png` and `out/pathtrace.png` from the default flags, the panel of each, and the images that the write-ups ask for.
- **Three write-ups**: answer from your own measurements and the equations here, not from a general description of the topic. Several questions disagree with the chapter on purpose. A number that contradicts the chapter, reported honestly and explained, is a good answer. A number that agrees with the chapter with no evidence is not.
- **AI prompt log**: one entry per prompt, each with the **prompt**, the **model**, the **idea behind it** (what you needed and what you already knew) and the **verification** (how you checked it: the reference images, a worked example by hand, the mean against the reference, a measurement).

A log entry like "wrote the path tracer for me" with empty idea and verification fields does not meet this requirement. You may ask AI to write a function. The write-ups and those two fields are where you show your own thinking. You should be able to answer a follow-up question

in person about anything you submit, and in this lab you will: the last ten minutes of the session are for that.

9 References

- Dendumrongkul, B. (2026): Ray Tracing vs. Path Tracing, the example page: the same scenes by a Whitted ray tracer, a path tracer and a bidirectional path tracer, with sliders.
- Sokolov, D. (2019): tinyraytracer, and its wiki: the lesson order and the material model of Part I.
- Whitted, T. (1980): *An Improved Illumination Model for Shaded Display*, Communications of the ACM 23(6). Part I.
- Möller, T., Trumbore, B. (1997): *Fast, Minimum Storage Ray-Triangle Intersection*, Journal of Graphics Tools 2(1). Equation 6.
- Wald, I. (2007): *On Fast Construction of SAH-based Bounding Volume Hierarchies*, IEEE Symposium on Interactive Ray Tracing. The binned SAH builder in `bvh.cpp`.
- Cook, R. L., Porter, T., Carpenter, L. (1984): *Distributed Ray Tracing*, SIGGRAPH '84. Section 5.
- Kajiya, J. T. (1986): *The Rendering Equation*, SIGGRAPH '86. Part III.
- Pharr, M., Jakob, W., Humphreys, G. (2023): *Physically Based Rendering*, 4th ed., Ch. 2 (Monte Carlo, Russian roulette), Ch. 7 (BVHs) and Ch. 13 (path tracing).
- Duff, T. et al. (2017): *Building an Orthonormal Basis, Revisited*, JCGT 6(1). `make_basis` in `vec3.h`.